

Java

A 4-page quick reference guide

1. Java Fundamentals

Java is a statically typed, compiled, object-oriented language that runs on the Java Virtual Machine (JVM), giving it "write once, run anywhere" portability.

Basic Syntax

```
public class Main {
    public static void main(String[] args) {
        int x = 10;
        double y = 3.14;
        String name = "Alice";
        boolean isValid = true;

        System.out.println("Hello, " + name);
    }
}
```

Control Flow

```
if (x > 5) {
    System.out.println("big");
} else if (x == 5) {
    System.out.println("equal");
} else {
    System.out.println("small");
}

for (int i = 0; i < 5; i++) {
    System.out.println(i);
}

while (x > 0) {
    x--;
}
```

2. Collections & Generics

The Java Collections Framework provides interfaces like List, Set, and Map, with generics giving compile-time type safety.

Common Collections

```
List<Integer> nums = new ArrayList<>();
nums.add(1);
nums.add(2);

Map<String, Integer> ages = new HashMap<>();
ages.put("Bob", 30);

Set<String> unique = new HashSet<>();
unique.add("x");
```

Iterating Collections

```
for (int n : nums) {
    System.out.println(n);
}

for (Map.Entry<String, Integer> e : ages.entrySet()) {
    System.out.println(e.getKey() + ": " + e.getValue());
}
```

Streams (Java 8+)

```
List<Integer> squares = nums.stream()
    .map(n -> n * n)
    .filter(n -> n > 1)
    .collect(Collectors.toList());
```

3. Object-Oriented Programming

Java enforces class-based OOP with strong typing, interfaces, and access modifiers.

```
public abstract class Animal {
    protected String name;

    public Animal(String name) {
        this.name = name;
    }

    public abstract String speak();
}

public class Dog extends Animal {
    public Dog(String name) {
        super(name);
    }

    @Override
    public String speak() {
        return name + " says Woof!";
    }
}
```

Key Concepts

- **Interfaces** — define a contract of methods a class must implement
- **Access modifiers** — public, private, protected, and package-private control visibility
- **Polymorphism** — a parent reference can point to a child object
- **Exception handling** — checked vs. unchecked exceptions via try/catch/finally

Exception Handling

```
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    System.out.println("Done");
}
```

4. Tooling & Best Practices

Build Tools

- **Maven** — declarative XML-based dependency & build management (pom.xml)
- **Gradle** — flexible, script-based build tool using Groovy or Kotlin DSL
- **javac / java** — core compiler and runtime commands for quick builds

Example pom.xml Snippet

```
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Best Practices

- Follow standard naming conventions (PascalCase classes, camelCase methods)
- Favor composition and interfaces over deep inheritance chains
- Use try-with-resources for closing streams/connections automatically
- Write unit tests with JUnit and aim for meaningful coverage
- Use immutable objects and final fields where possible for thread safety